

02-STL提高

一、本阶段目标

这一章主要补充 STL 中更常用、但比 `vector` / `map` / `set` 稍微进阶一点的工具。

重点不是背语法，而是知道：

- 什么情况用队列
 - 什么情况用栈
 - 什么情况用优先队列
 - 什么情况用 `lower_bound` / `upper_bound`
 - 常见 `algorithm` 函数怎么写
 - 哪些地方容易越界或写错
-

二、知识点总览

本章包括：

- `queue` 队列
 - `stack` 栈
 - `deque` 双端队列
 - `priority_queue` 优先队列
 - `lower_bound`
 - `upper_bound`
 - 常用 `algorithm` 函数
 - STL 容器选择表
-

三、`queue` 队列

1. `queue` 是什么？

`queue` 是队列。

特点是：

| 先进先出

也就是：

```
1 先进去的，先出来。
```

可以想成排队买饭。

2. 头文件

```
1 #include<queue>
```

3. 定义

```
1 queue<int> q;
```

4. 常用操作

```
1 q.push(x);      // 入队
2 q.front();      // 查看队首元素
3 q.pop();        // 删除队首元素
4 q.empty();      // 判断是否为空
5 q.size();       // 元素个数
```

注意：

```
1 front() 和 pop() 前，要保证队列不是空的。
```

5. 基础模板

```
1 #include<iostream>
2 #include<queue>
3 using namespace std;
4
5 int main()
6 {
```

```
7     queue<int> q;
8
9     q.push(1);
10    q.push(2);
11    q.push(3);
12
13    while(!q.empty())
14    {
15        cout<<q.front()<<" ";
16        q.pop();
17    }
18
19    return 0;
20 }
```

输出：

```
1  1 2 3
```

6. 常见用途

queue 常用于：

- BFS
- 按顺序处理任务
- 模拟排队
- 层序遍历

后面搜索专题会大量用到。

四、stack 栈

1. stack 是什么？

stack 是栈。

特点是：

| 后进先出

也就是：

```
1    最后进去的，最先出来。
```

可以想成一摞盘子。

2. 头文件

```
1    #include<stack>
```

3. 定义

```
1    stack<int> st;
```

4. 常用操作

```
1    st.push(x);      // 入栈
2    st.top();        // 查看栈顶元素
3    st.pop();        // 删除栈顶元素
4    st.empty();      // 判断是否为空
5    st.size();       // 元素个数
```

注意：

```
1    top() 和 pop() 前，要保证栈不是空的。
```

5. 基础模板

```
1    #include<iostream>
2    #include<stack>
3    using namespace std;
4
5    int main()
6    {
```

```
7     stack<int> st;
8
9     st.push(1);
10    st.push(2);
11    st.push(3);
12
13    while(!st.empty())
14    {
15        cout<<st.top()<<" ";
16        st.pop();
17    }
18
19    return 0;
20 }
```

输出：

```
1   3 2 1
```

6. 常见用途

stack 常用于：

- 括号匹配
- 表达式处理
- DFS 的非递归写法
- 单调栈

五、deque 双端队列

1. deque 是什么？

deque 是双端队列。

特点是：

两头都可以插入和删除。

2. 头文件

```
1  #include<deque>
```

3. 定义

```
1  deque<int> dq;
```

4. 常用操作

```
1  dq.push_back(x);      // 从后面加入
2  dq.push_front(x);     // 从前面加入
3  dq.pop_back();        // 删除最后一个
4  dq.pop_front();       // 删除第一个
5  dq.front();           // 查看第一个
6  dq.back();            // 查看最后一个
7  dq.empty();           // 判断是否为空
8  dq.size();            // 元素个数
```

5. 基础模板

```
1  #include<iostream>
2  #include<deque>
3  using namespace std;
4
5  int main()
6  {
7      deque<int> dq;
8
9      dq.push_back(1);
10     dq.push_back(2);
11     dq.push_front(0);
12
13     while(!dq.empty())
14     {
15         cout<<dq.front()<<" ";
16         dq.pop_front();
```

```
17     }
18
19     return 0;
20 }
```

输出：

```
1  0 1 2
```

6. 常见用途

deque 常用于：

- 双端操作
- 滑动窗口
- 单调队列
- 需要两头都能操作的模拟题

六、priority_queue 优先队列

1. priority_queue 是什么？

priority_queue 是优先队列。

可以理解为：

每次都能快速取出当前优先级最高的元素。

默认情况下，C++ 的 priority_queue 是大根堆。

也就是说：

```
1  top() 取出来的是当前最大的元素。
```

2. 头文件

```
1  #include<queue>
```

3. 大根堆

```
1  priority_queue<int> q;
```

示例：

```
1  #include<iostream>
2  #include<queue>
3  using namespace std;
4
5  int main()
6  {
7      priority_queue<int> q;
8
9      q.push(3);
10     q.push(1);
11     q.push(5);
12
13     cout<<q.top();
14
15     return 0;
16 }
```

输出：

```
1  5
```

4. 常用操作

```
1  q.push(x);      // 加入元素
2  q.top();        // 查看堆顶元素
3  q.pop();        // 删除堆顶元素
4  q.empty();      // 判断是否为空
5  q.size();       // 元素数量
```

注意：

```
1  top() 和 pop() 前，要保证队列不是空的。
```

5. 小根堆

如果想每次取出最小值：

```
1  priority_queue<int, vector<int>, greater<int>> q;
```

完整代码：

```
1  #include<iostream>
2  #include<queue>
3  #include<vector>
4  #include<functional>
5  using namespace std;
6
7  int main()
8  {
9      priority_queue<int, vector<int>, greater<int>> q;
10
11      q.push(3);
12      q.push(1);
13      q.push(5);
14
15      cout<<q.top();
16
17      return 0;
18  }
```

输出：

```
1  1
```

6. 小根堆记忆方式

```
1  priority_queue<int, vector<int>, greater<int>> q;
```

可以拆成三部分看：

```
1  int: 里面存 int
2  vector<int>: 底层用 vector 存
3  greater<int>: 小的优先
```

7. priority_queue 常见用途

常用于：

- 每次取最大值
- 每次取最小值
- 合并果子
- Dijkstra 最短路
- 动态维护当前最优元素

典型题：

```
1 P1090 合并果子
```

8. priority_queue 易错点

错误 1：以为默认是小根堆

错误理解：

```
1 priority_queue 默认取最小值
```

正确理解：

```
1 priority_queue 默认取最大值
```

错误 2：pop 会返回元素

错误写法：

```
1 int x=q.pop();
```

原因：

```
1 pop() 只负责删除，不返回值。
```

正确写法：

```
1  int x=q.top();
2  q.pop();
```

错误 3：空队列 top

错误写法：

```
1  cout<<q.top();
```

正确写法：

```
1  if(!q.empty())
2  {
3      cout<<q.top();
4  }
```

七、lower_bound 和 upper_bound

1. 头文件

```
1  #include<algorithm>
```

2. 使用前提

一般使用前要保证：

```
1  数组或 vector 已经排好序。
```

也就是先：

```
1  sort(a.begin(),a.end());
```

再用：

```
1  lower_bound
2  upper_bound
```

3. lower_bound 是什么？

```
1 lower_bound(a.begin(),a.end(),x)
```

意思是：

```
1 找到第一个 >= x 的位置。
```

4. upper_bound 是什么？

```
1 upper_bound(a.begin(),a.end(),x)
```

意思是：

```
1 找到第一个 > x 的位置。
```

5. 一句话记忆

```
1 lower_bound: 第一个 >= x  
2 upper_bound: 第一个 > x
```

也可以这样记：

```
1 lower: 从 x 本身开始找  
2 upper: 从 x 上面开始找
```

6. 在 vector 中使用

```
1 vector<int> a={1,2,2,2,5,7};  
2  
3 auto it1=lower_bound(a.begin(),a.end(),2);  
4 auto it2=upper_bound(a.begin(),a.end(),2);
```

此时：

```
1 it1 指向第一个 2
2 it2 指向 5
```

7. 求下标

`lower_bound` 和 `upper_bound` 返回的是迭代器，不是下标。

如果要下标：

```
1 int pos=lower_bound(a.begin(),a.end(),x)-a.begin();
```

完整例子：

```
1 #include<iostream>
2 #include<vector>
3 #include<algorithm>
4 using namespace std;
5
6 int main()
7 {
8     vector<int> a={1,2,2,2,5,7};
9
10    int x=2;
11
12    int l=lower_bound(a.begin(),a.end(),x)-a.begin();
13    int r=upper_bound(a.begin(),a.end(),x)-a.begin();
14
15    cout<<l<<" "<<r;
16
17    return 0;
18 }
```

输出：

```
1 1 4
```

8. 统计 x 出现次数

```
1 int cnt=upper_bound(a.begin(),a.end(),x)-lower_bound(a.begin(),a.end(),x);
```

例如：

```
1  vector<int> a={1,2,2,2,5,7};
2
3  int cnt=upper_bound(a.begin(),a.end(),2)-lower_bound(a.begin(),a.end(),2);
```

结果：

```
1  3
```

9. 判断 x 是否存在

```
1  int pos=lower_bound(a.begin(),a.end(),x)-a.begin();
2
3  if(pos<a.size() && a[pos]==x)
4  {
5      cout<<"存在";
6  }
7  else
8  {
9      cout<<"不存在";
10 }
```

注意：

```
1  一定要先判断 pos<a.size()
```

否则如果 `x` 比所有数都大，`pos` 会等于 `a.size()`，这时访问 `a[pos]` 会越界。

10. 统计一个范围内有多少个数

如果要统计 `[L,R]` 中有多少个数：

```
1  int cnt=upper_bound(a.begin(),a.end(),R)-lower_bound(a.begin(),a.end(),L);
```

这个非常重要。

例如：

```
1  vector<int> a={1,2,3,5,7,9};
2
3  int L=3,R=7;
4
5  int cnt=upper_bound(a.begin(),a.end(),R)-lower_bound(a.begin(),a.end(),L);
```

统计的是：

```
1  3 5 7
```

所以：

```
1  cnt = 3
```

11. 在普通数组中使用

```
1  int a[100];
2
3  sort(a,a+n);
4
5  int pos=lower_bound(a,a+n,x)-a;
```

完整例子：

```
1  #include<iostream>
2  #include<algorithm>
3  using namespace std;
4
5  int main()
6  {
7      int n,x;
8      cin>>n>>x;
9
10     int a[100];
11
12     for(int i=0;i<n;i++)
13     {
14         cin>>a[i];
15     }
16
17     sort(a,a+n);
```

```
18
19     int pos=lower_bound(a,a+n,x)-a;
20
21     cout<<pos;
22
23     return 0;
24 }
```

12. 和 Forbidden List2 的关系

Forbidden List2 的核心是：

- 1 在有序的禁止列表中，快速统计某一段范围内有多少个禁止数字。

常见写法：

```
1  int cnt=upper_bound(forbid.begin(),forbid.end(),r)
2      -lower_bound(forbid.begin(),forbid.end(),l);
```

含义：

- 1 统计 $[l, r]$ 中有多少个禁止数字。

八、常用 algorithm 函数

1. sort

```
1  sort(a.begin(),a.end());
```

作用：

- 1 从小到大排序。

普通数组：

```
1  sort(a,a+n);
```

注意：


```
1  sort 会放到排序专题里详细讲。
```

2. reverse

```
1  reverse(a.begin(),a.end());
```

作用：

```
1  反转数组或 vector。
```

示例：

```
1  vector<int> a={1,2,3};  
2  
3  reverse(a.begin(),a.end());
```

结果：

```
1  3 2 1
```

3. max 和 min

```
1  max(a,b);  
2  min(a,b);
```

示例：

```
1  int x=max(3,5);  
2  int y=min(3,5);
```

结果：

```
1  x = 5  
2  y = 3
```

4. swap

```
1 swap(a,b);
```

作用：

```
1 交换两个变量的值。
```

示例：

```
1 int a=3,b=5;  
2  
3 swap(a,b);
```

结果：

```
1 a = 5  
2 b = 3
```

5. count

```
1 count(a.begin(),a.end(),x);
```

作用：

```
1 统计 x 出现了几次。
```

示例：

```
1 vector<int> a={1,2,2,3};  
2  
3 int cnt=count(a.begin(),a.end(),2);
```

结果：

```
1 cnt = 2
```

注意：

```
1 algorithm 里的 count 是直接遍历统计，复杂度是  $O(n)$ 。
```

如果数组已经有序，用：

```
1 upper_bound - lower_bound
```

通常更适合多次查询。

6. find

```
1 find(a.begin(), a.end(), x);
```

作用：

```
1 查找 x 是否存在。
```

示例：

```
1 vector<int> a={1,2,3};
2
3 auto it=find(a.begin(),a.end(),2);
4
5 if(it!=a.end())
6 {
7     cout<<"存在";
8 }
9 else
10 {
11     cout<<"不存在";
12 }
```

注意：

```
1 find 找不到时，返回 a.end()。
```

7. fill

```
1 fill(a,a+n,0);
```

作用：

- 1 把数组某一段全部赋成同一个值。

普通数组：

```
1 int a[100];
2
3 fill(a,a+100,0);
```

vector：

```
1 vector<int> a(100);
2
3 fill(a.begin(),a.end(),0);
```

8. unique

```
1 unique(a.begin(),a.end());
```

作用：

- 1 把相邻重复元素移动到后面，返回去重后末尾位置。

注意：

- 1 unique 通常要先 sort。

标准写法：

```
1 sort(a.begin(),a.end());
2
3 a.erase(unique(a.begin(),a.end()),a.end());
```

示例：

```
1 vector<int> a={3,1,2,3,2};
2
3 sort(a.begin(),a.end());
4
5 a.erase(unique(a.begin(),a.end()),a.end());
```

结果：

```
1 1 2 3
```

九、常见容器选择表

需求	推荐工具
存一串数	vector
存二维矩阵	vector<vector>
统计次数	map
自动去重	set
先进先出	queue
后进先出	stack
两头都要操作	deque
每次取最大值	priority_queue
每次取最小值	小根堆 priority_queue
有序数组中找边界	lower_bound / upper_bound
查找元素是否存在	find / count / set.count / map.count
排序	sort
去重	sort + unique / set

十、常见易错点总结

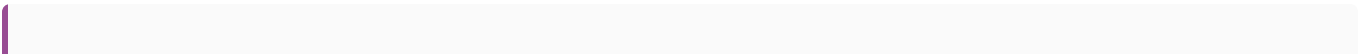
1. top / front / back / pop 前要判空

错误：

```
1 cout<<q.front();
```

如果 q 是空的，会出问题。

正确：



```
1  if(!q.empty())
2  {
3      cout<<q.front();
4  }
```

2. pop 不返回元素

错误：

```
1  int x=q.pop();
```

正确：

```
1  int x=q.front();
2  q.pop();
```

对于 `priority_queue`：

```
1  int x=q.top();
2  q.pop();
```

3. lower_bound 返回的是迭代器

错误理解：

```
1  lower_bound 直接返回下标。
```

正确：

```
1  int pos=lower_bound(a.begin(),a.end(),x)-a.begin();
```

4. lower_bound 可能返回 end

所以访问前必须判断：

```
1  if(pos<a.size() && a[pos]==x)
```

5. priority_queue 默认是大根堆

默认：

```
1 priority_queue<int> q;
```

取出最大值。

小根堆：

```
1 priority_queue<int,vector<int>,greater<int>> q;
```

6. unique 不会直接删除元素

错误理解：

```
1 unique 会直接把 vector 变短。
```

正确理解：

```
1 unique 只是把重复元素移动到后面，并返回新的结尾位置。
```

真正删除要写：

```
1 a.erase(unique(a.begin(),a.end()),a.end());
```

十一、本专题对应的题目

题号 / 题目	类型	题面简述	对应知识点
P2249 查找	二分查找	在有序数组中查询某个数第一次出现的位置	lower_bound
P1102 A-B 数对	排序 / 二分	统计满足 $A-B=C$ 的数对数量	lower_bound / upper_bound
P1090 合并果子	贪心 / 堆	每次合并最小的两堆果子，求最小总代价	priority_queue 小根堆

题号 / 题目	类型	题面简述	对应知识点
Forbidden List2	AtCoder / 二分	给定禁止数字列表，询问从 X 开始第 Y 个非禁止数字	lower_bound / upper_bound
BFS 类题	搜索	按层扩展状态	queue
括号匹配类题	栈	判断括号是否合法匹配	stack

十二、本章最重要的结论

这一章不是为了把所有 STL 都背下来。

最重要的是：

- 1 看到题目时，能想到应该用什么工具。

例如：

- 1 要按顺序处理：queue
- 2 要反向处理：stack
- 3 要两头操作：deque
- 4 要每次取最小/最大：priority_queue
- 5 要有序查边界：lower_bound / upper_bound
- 6 要去重：set 或 sort + unique
- 7 要统计次数：map